

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

2. JavaScript ES6

8. ES6 JS I: 語法I

8. ES6 JS I: 語法I

學習目標

- 了解什麼是ES6，以及它是如何解決ES5的問題。
- 了解ES6的新功能，以及如何使用它們。

8. ES6 JS I: 語法I

內容

8.1
ES6簡介

8.2
ES6語法一

8.3
默認參數

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

2. JavaScript ES6

8.1 ES6 簡介

8. ES6 JS I: 語法I

什麼是ES6？

- 也被稱為ECMAScript 6 / ECMAScript 2015。
- 它是一種編程語言標準(如Jscript, ActionScript)。
- 建立ES6是為了規範JavaScript。
- 使得JS程式碼更加現代化和可讀。

8. ES6 JS I: 語法I

ES6 vs ES5

- ES5是ECMAScript的第五版，ES6是ES5的下一個主要改進。
- ES6有新的運算和方法，可以大大降低使用ES5的程式碼的複雜性。
- 很多ES5的程式碼都沒有更新到ES6中。

8. ES6 JS I: 語法I

ES6的使用

- ES6 JS的運作方式與舊版本的JS類似，但加入了新的語法、函式和功能，使建立複雜的邏輯和編寫網站更容易。
- ES6的新增功能也解決了一些舊版本的問題。

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

2. JavaScript ES6

8.2 ES6語法一

8. ES6 JS I: 語法I

let vs var

- 在ES6之前，var關鍵字是用來宣告變數的。
- 所有使用var宣告的變數都被加入到全域物件 - window的屬性列表中。

```
var variable_name;
```

```
var x = 50;  
console.log(window.x) // 50
```

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

let vs var

- ES6新增了let關鍵字來宣告變數。
- 使用let宣告的變數不會附加到全域物件上。

```
let x = 50;  
console.log(window.x) // undefined
```

8. ES6 JS I: 語法I

let 區塊範圍(block-scoping)

- 使用let關鍵字宣告的變數是有區塊範圍(block-scoped)的。
- 區塊是一對大括號{ }內的空間。

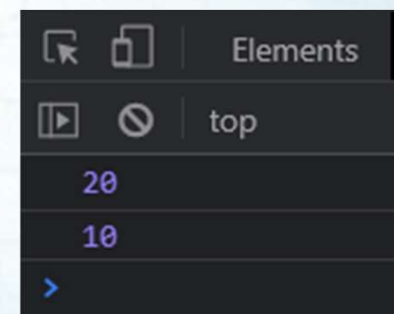
```
<script>
  let x = 10;
  if (x == 10) {
    // This is a block
  };
  // This is outside the block
</script>
```

8. ES6 JS I: 語法I

let 區塊範圍(block-scoping)

- 在右邊的這個例子中，區塊內的x是一個新變數，與區塊外宣告的x不同。
- 程式碼內部的x將是20，而程式碼外部的x將是10。

```
<script>
  let x = 10;
  if (x == 10) {
    x = 20;
    console.log(x);
  };
  console.log(x);
</script>
```



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

for迴圈(for loop)中的let

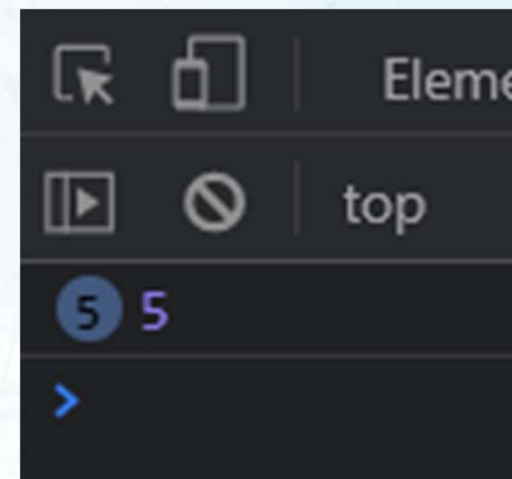
- 右邊的例子顯示了for迴圈(for loop)。
- 目的是每秒鐘輸出從0到4的數字。

```
for (var i = 0; i < 5; i++) {  
  setTimeout(function () {  
    console.log(i);  
  }, 1000);  
};
```

8. ES6 JS I: 語法I

for迴圈(for loop)中的let

- 執行該程式碼將導致...
- 輸出“5”這個數字五次。
- 為什麼會這樣？
- 要知道答案，需要深入了解JS的非同步性。



8. ES6 JS I: 語法I

非同步的(Asynchronous) JS

- JavaScript引擎會從第一句程式碼開始執行，直到最後一句。
- `setTimeout`是一個特別的函式，在這個例子中，用來設定在等待1秒後調用一個名為`sayHi`程式區塊。

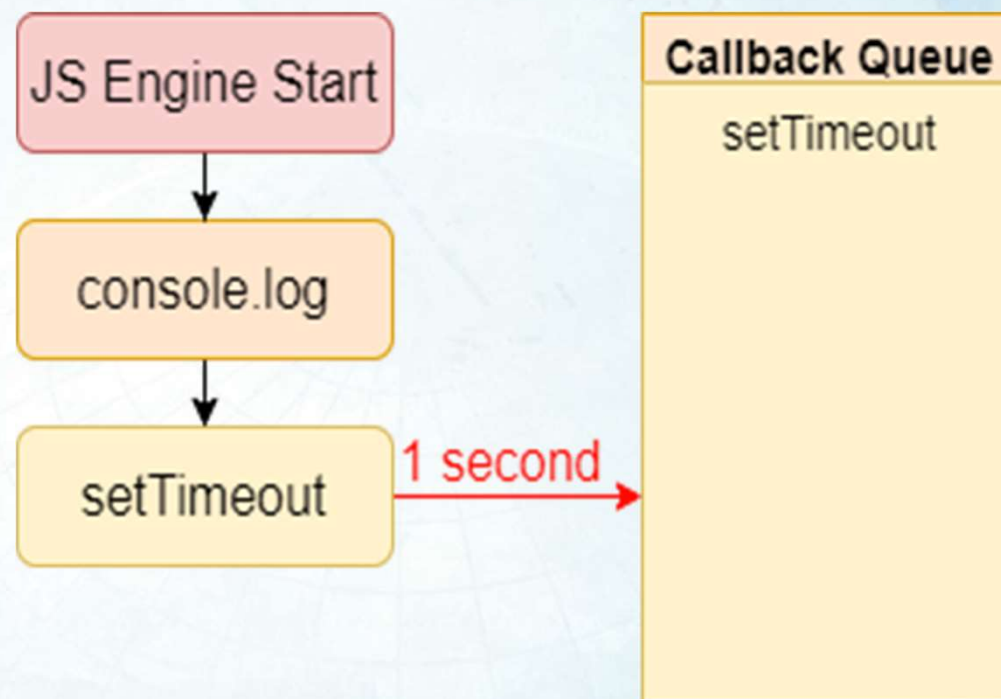
```
<script>  
  console.log("first statement");  
  
  setTimeout( function sayHi() {  
    console.log('Hi')  
  }, 1000);  
  
  console.log("last statement");  
</script>
```

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

非同步的(Asynchronous) JS

- 當JS引擎進入setTimeout函式時，它會立即建立一個計時器，等待1秒的時間。
- 當時間結束後，該函式會被傳遞到回調隊列(callback queue)中。

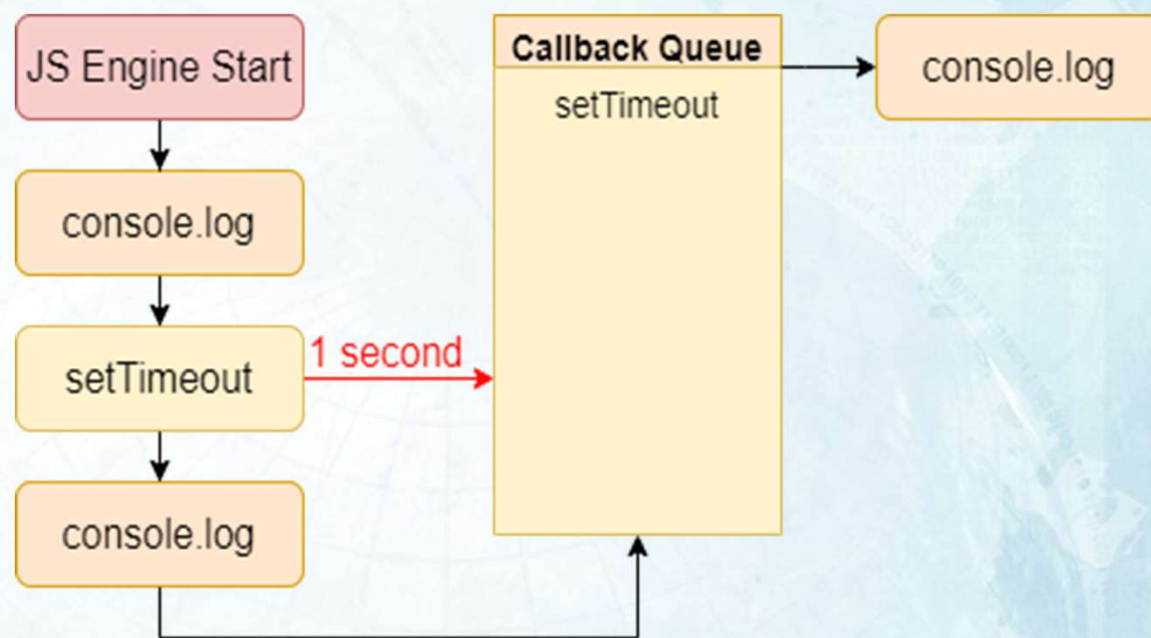


All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

非同步的(Asynchronous) JS

- 在執行呼叫堆疊(call stack)中的所有程式碼(假設這些程式碼不在setTimeout函式內)完成後，
- 會開始按照順序執行回調隊列(callback queue)中的程式碼。

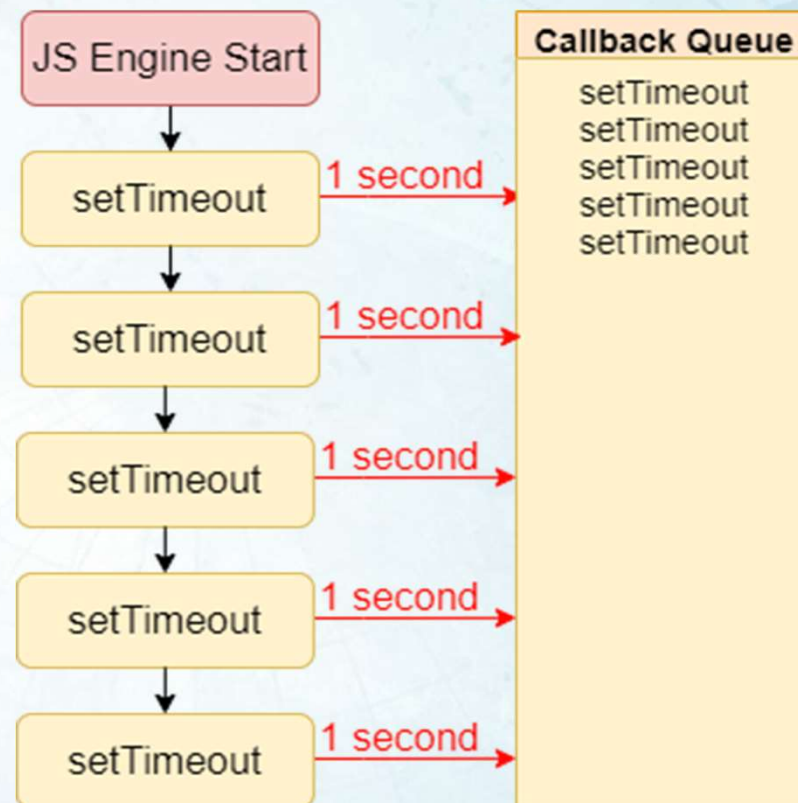


All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

for迴圈(for loop)中的let

- 回到for迴圈的例子。
- 在第一次迴圈中，調用了setTimeout並建立了一個定時器，等待1秒的時間。
- 在不執行任何console.log的情況下，i 的值會加 1，並且迴圈會繼續執行，總共執行了5次。
- 注意，所有的定時器都是同時發生的，因為它是非同步的。

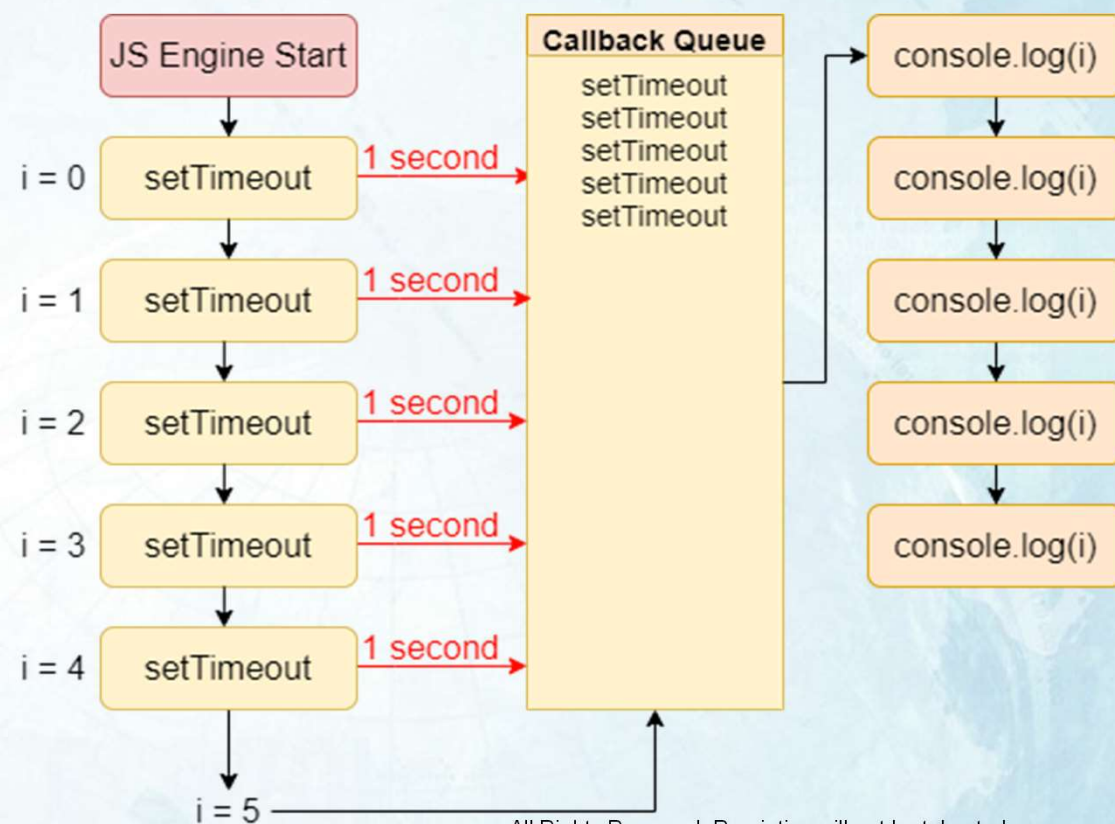


All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

for迴圈(for loop)中的let

- 在所有5個setTimeout等待完畢後，回調隊列(callback queue)中的程式碼會按照順序執行。
- 所有的回調隊列都使用 i，這時 i 的值已經是5了(i 的值在每個循環結束時被添加1)，因此所有的 console.log 都會顯示5。



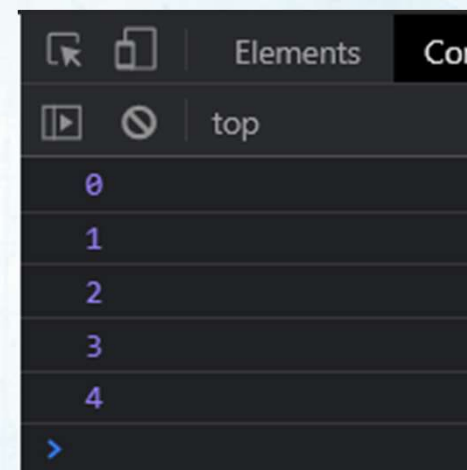
All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

for迴圈(for loop)中的let

- 要解決這個問題，只需將var改為let。
- let與var不同的是，let是有區塊範圍的(block scoped)，所以這個值不會被到處使用，只有在被宣告的區塊內使用。

```
for (let i = 0; i < 5; i++) {  
  setTimeout(function () {  
    console.log(i);  
  }, 1000);  
};
```

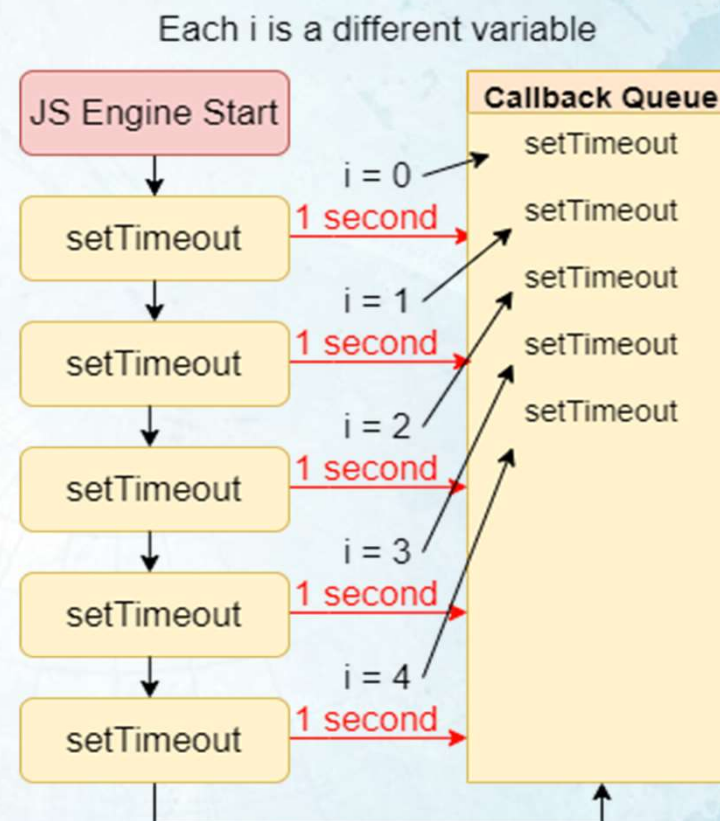


All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

for迴圈(for loop)中的let

- 使用相同的流程邏輯。
- **let**在每個迴圈中宣告一個新的 **i**，並且只在該迴圈中使用。
- 有5個迴圈，所以會有5個 **i**，每個 **i** 的值是從0到4。
- 當每個**setTimeout**執行時，會使用在其迴圈中宣告的 **i**，產生所需的效果。



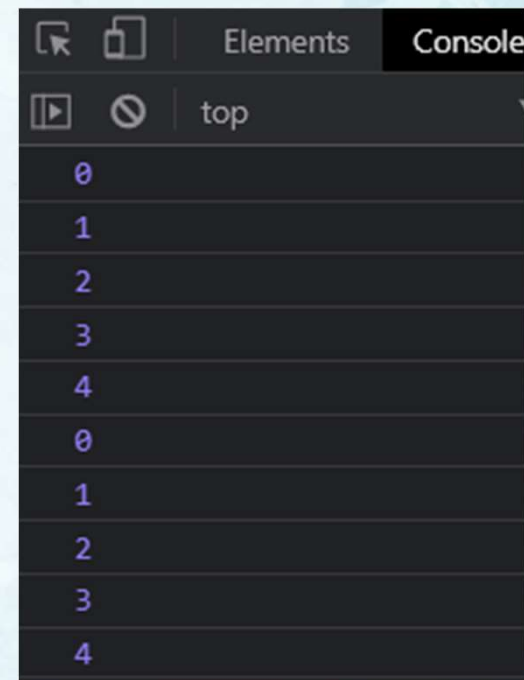
All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

for迴圈(for loop)中的let

- 需要注意的是，這只會出現在像 `setTimeout` 這類非同步函式中。
- 對於其他函式，例如 `console.log`，使用 `var` 或 `let` 不會有區別，因為沒有函式會被放入回調隊列(callback queue)中。

```
for (var i = 0; i < 5; i++) {  
    console.log(i);  
};  
  
for (let i = 0; i < 5; i++) {  
    console.log(i);  
};
```



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

let vs var

- **let**和**var**的一個區別在於範圍(scope)。
- 如前所述，使用**let**宣告的變數是有區塊範圍的(block-scoped)。
- 而**var**宣告的變數通常是全局的，除非是在函式內部宣告的。
- 這意味著，如果**var**變數是在函式內部宣告的，就無法在函式外被引用。

```
function funct1() {  
    var i = 10;  
}  
console.log(i);
```

✖ ▶ Uncaught ReferenceError: i is not defined

8. ES6 JS I: 語法I

let vs var

- 需要注意的是，**var**只有在函式內部才是局部變數。在**for**迴圈、**if-else**語句以及其他語句中使用的**var**，只要不在函式內部，就會是全局變數。
- 而**let**宣告的變數只能在區塊(block)內部存取，這對於**if-else**語句和**for**迴圈也適用。

```
let x = 10;  
if (x == 10) {  
    let y = 20;  
};  
console.log(y);
```

✖ Uncaught ReferenceError: y is not defined

8. ES6 JS I: 語法I

let vs var

- **var**允許重新宣告同一個變數，但不會有任何作用。
- **let**不能重新宣告同一個變數，否則只會產生錯誤。

```
var x = 20;  
var x;  
console.log(x); // 20
```

```
let x = 20;  
let x; // error  
console.log(x);
```

8. ES6 JS I: 語法I

let vs var

- 使用var宣告的變數會被提升(hoist)。
- 使用let宣告的變數則不能提升。
- 提升(Hoisting)是一個複雜的概念，之後會簡單解釋一下。

```
console.log(x);  
var x = 20;
```

```
console.log(x);  
let x = 20;
```

```
✖ ▶ Uncaught ReferenceError: Cannot access 'x' before  
initialization
```

8. ES6 JS I: 語法I

提升(Hoisting)

- 在右邊的例子中，有一段程式碼在變數宣告之前，就使用`console.log`輸出該變數。
- 使用`var`是可以這樣做的。
- 提升(Hoisting)是一種行為，看起來像是將變數宣告移動到最上面。
- 但實際上並不會真的發生移動。

```
console.log(x);  
var x = 20;
```

相當於

```
var x;  
console.log(x);  
x = 20;
```

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

提升(Hoisting)

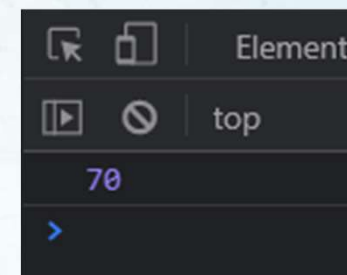
- 使用let這樣做的會產生錯誤。
- 不管是使用var或者let，最好還是先宣告變數，以避免混亂和潛在的錯誤。

8. ES6 JS I: 語法I

常數(Constants)

- 常數關鍵字const的作用與let關鍵字類似。
- 宣告的變數也是有區塊範圍的(block-scoped)。
- 不過這個變數是唯讀的，不能重新賦值。
- 而由let關鍵字宣告的變數，則可以重新賦值。

```
let MARKS = 100;  
MARKS -= 30;  
console.log(MARKS);
```



8. ES6 JS I: 語法I

常數(Constants)

- `const`宣告的變數必須被使用並賦值。
- 變數的值也是不可改變的。
- 試圖不賦值或者重新賦值到`const`變數將會導致`TypeError`。

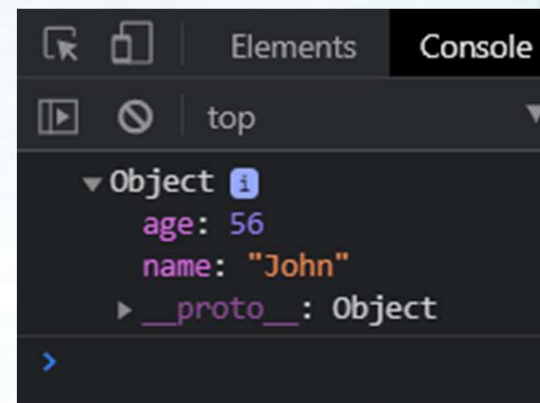
```
const MARKS = 100;  
MARKS -= 30;  
console.log(MARKS);
```

✖ ▶ Uncaught TypeError: Assignment to constant variable.

8. ES6 JS I: 語法I

常數(Constants)與物件(Objects)

- 物件宣告為常數變數時，包含的是一個指標(指向該物件在記憶體中的位置)。
- 改變物件的屬性值，不會改變變數的指標，是允許的。
- 但將物件重新分配給另一個物件，會改變變數的指標，是不被允許的。



```
const PERSON = {  
  name: "John",  
  age: 55  
};  
PERSON.age = 56;  
console.log(PERSON);
```

```
const PERSON = {  
  name: "John",  
  age: 55  
};  
PERSON = {  
  name: "John",  
  age: 56  
};  
console.log(PERSON);
```

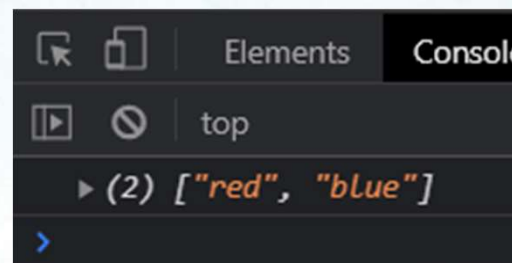
✖ ▶ Uncaught TypeError: Assignment to constant variable.

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

常數(Constants)與陣列(Arrays)

- 如果陣列被分配為常數。
- 像push和pop這樣的陣列方法仍然允許被使用。
- 但重新分配陣列將會導致TypeError。



```
const COLORS = ["red"];  
COLORS.push("blue");  
console.log(COLORS);
```

```
COLORS = []  
console.log(COLORS);
```

✖ ▶ Uncaught TypeError: Assignment to constant variable.

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

常數的普遍做法

- 在前面的例子中，宣告的常數變數都是用大寫字母的。
- 雖然沒有必要，但使用大寫字母和下劃線來聲明變數是好的做法。
- 這使得它更容易閱讀，並與使用`let`關鍵字的變數不同。

8. ES6 JS I: 語法I

常數的普遍做法

- 在這個例子，可以將常數用作分配顏色程式碼。
- 這將使在需要調用顏色時更容易輸入和閱讀。

```
const COLOR_RED = "#E72019";  
const COLOR_GREEN = "#05D731";  
const COLOR_BLUE = "#0E31E2";
```

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

2. JavaScript ES6

8.3 默認參數(Default Parameter)

8. ES6 JS I: 語法I

參數(Parameters)和引數(Arguments)

- 參數和引數經常互換使用。
- 在右邊的例子中，
- 添加函式有2個參數: x和y。
- 調用加法函式需要2個引數: 10和5。

```
function add(x, y) {  
    console.log(x + y)  
};  
  
add(10, 5);
```


8. ES6 JS I: 語法I

參數(Parameters)和引數(Arguments)

- 函式參數是未定義(undefined)默認值的。
- 如果不為函式提供引數，就會導致整個函式也未定義，而失去作用。

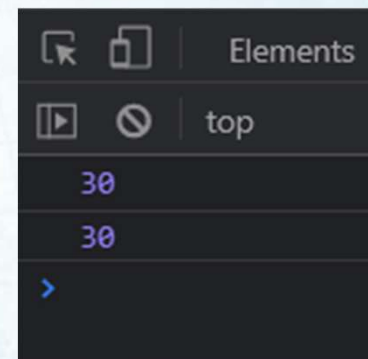
```
function add(x, y) {  
    console.log(x + y) // undefined + undefined  
};  
  
add();
```

8. ES6 JS I: 語法I

默認參數

- 只要設定函式參數的默認值，這樣即使沒有提供任何引數，該函式仍然可以發揮作用。
- 要做到這一點，就要在函式中為參數賦值。

```
function add(x=10, y=20) {  
    console.log(x + y) // 10 + 20  
};  
  
add();  
add(undefined, undefined);
```



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

默認參數練習

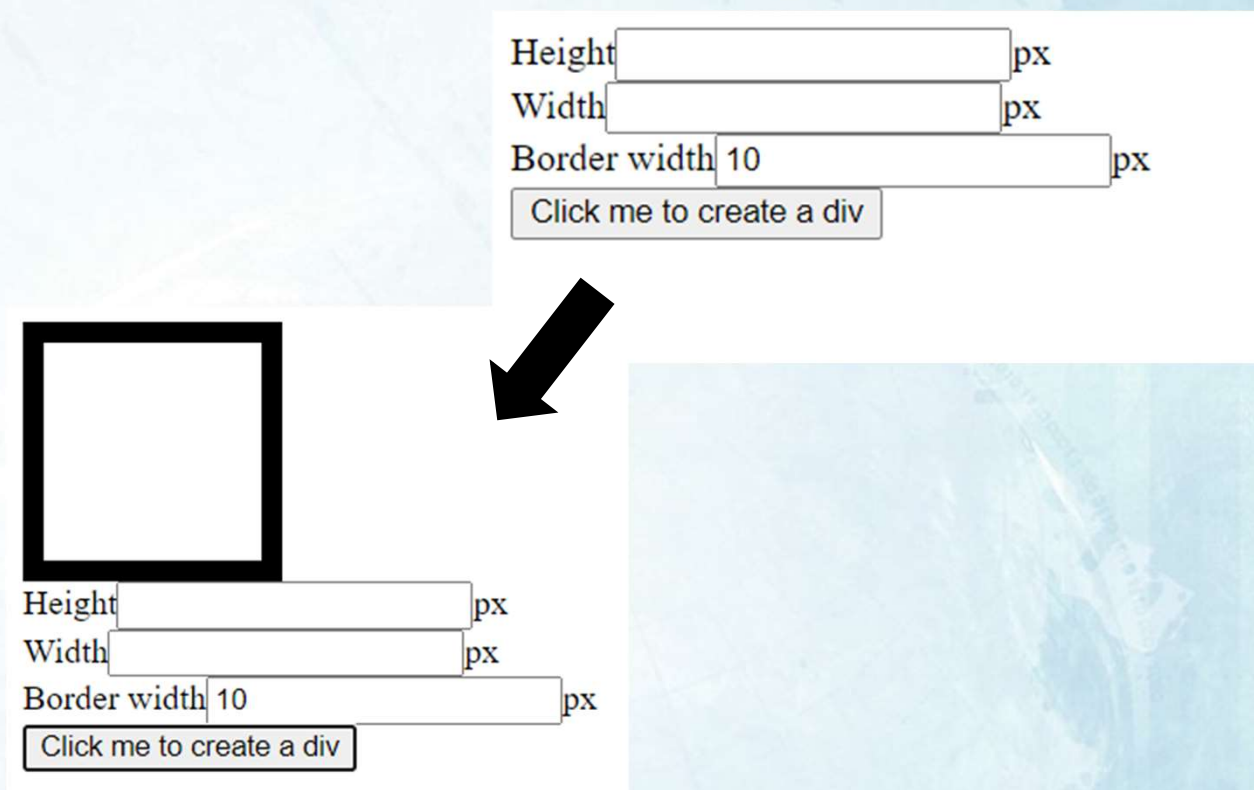
- 建立一個網站...
- 當中，有三個輸入框，分別是高度、寬度和邊框寬度。
- 此外，還有一個按鈕，在點擊按鈕時會調用一個函式。
- 該函式將根據輸入內容在<body>的最頂端建立一個div，並有三個參數：
 - height: div的高度，默認為100px
 - width: div的寬度，默認為100px
 - border-width: div邊框的寬度，默認為3px

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

默認參數練習示例

- 網頁給出3個輸入框和一個按鈕，如果任何一個輸入框輸入了有效數字 (例如在邊框寬度中輸入10)，並點擊按鈕。
- 將顯示建立的邊框寬度為10px的div。



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

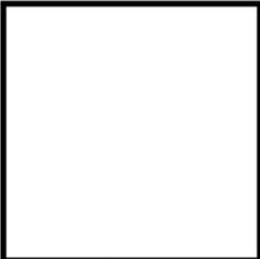
8. ES6 JS I: 語法I

默認參數練習示例

- 如果沒有輸入任何值，並點擊按鈕，那麼這個div就有100px高度、100px寬度和3px邊框寬度。

Height px
Width px
Border width px





Height px
Width px
Border width px

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究。

8. ES6 JS I: 語法I

參考

- 如果需要更多的解釋，請使用:
- <https://www.javascripttutorial.net/es6/>
- <https://javascript.info/>
- 如果需要更具體的答案，請使用:
- <https://stackoverflow.com/>